Phishing Detection using Random Forest & XGBoost

Nathan Perez
University of Texas Rio Grande Valley
CSCI 4341-04
Mission, Tx, United States
nathan.perez02@utrgv.edu

Amando Ramirez
University of Texas Rio Grande Valley
CSCI 4341-04
Brownsville, Tx, United States
amando.ramirez02@utrgv.edu

Roberto Molina Jr.
University of Texas Rio Grande Valley
CSCI 4341-04
Brownsville, Tx, United States
roberto.molina02@utrgv.edu

***Abstract*— This project is designed to use two machine learning models to detect phishing attacks, most prominently, URL phishing scams. XGBoost and Random Forest are the two machine learning algorithms that will be implemented, which leads to the overarching research question. Which machine learning model is superior? In this comparative study, the models will perform the same tasks and based on the observations made on their performance, a conclusion will be made as to which algorithm was more optimal. The datasets have been selected; the tool choices have been determined, and the results hold room for conversation as to what exactly each model's strong suits are. This paper reflects the structure of the projects design, as well as the results of how each model performed given the metrics of the project's scope.**

Keywords— phishing, XGBoost, Random Forest, dataset, URL, machine learning

I. Introduction

This project's goal was to detect phishing attacks using both XGBoost and Random Forest models. These tree-based algorithms are the foundation of this project and will be compared in the research question formulation.

They will use the same datasets and tools with variation only when necessary for the specific algorithm to eliminate any types of discrepancies that may appear to keep the comparisons consistent. This final report documents final results, comparisons, and efficiency of the selected learning language models to conclude which model is best suited to identify phishing attempts through URLs.

II. Background

A. Phishing

Phishing attacks have seen an influx of growth in recent years due to how technology oriented the world is today, and with no sign of stopping anytime soon, people must adapt to the ever-growing threats that come with the advancement of technology. [1] According to the U.S Department of State, "Phishing is a type of cyber-attack where cybercriminals attack you through social engineering, which involves deceptive communications designed to gain trust or elicit fear." Phishing URL links are a popular domain among scammers and can affect anyone, with many people oblivious to what can be legitimate and what can be considered a scam. As a result, many people's data is at risk, therefore minimizing the risk is a top priority and determining what is a potential danger.

B. Design

The overall design of the project involves selecting two learning language models to train with same datasets. These training sessions would then lead to an evaluation stage where the LLMs would attempt to identify and label URLs as either benign or as Phishing attempts. The results would be recorded and used to assess their levels of overall accuracy, precision, f1-score, recall, and levels of efficiency.

Figure 1. Design Flowchart

C. Machine Learning Approach

Two models will be supervised: XGBoost & Random Forest. These two machine learning models were selected due to the similarities, primarily their construction of trees to formulate decisions. Datasets were selected to train and test the LLMs in different ways to measure their performance against each other.

D. Random Forest

Random Forest builds an ensemble of decision trees with each tree utilizing bagging which trains each tree on a sample of data. The result leads to a final robust model because of the bagging method reducing variance and handling overfitting.

E. XGBoost

XGBoost is a gradient boosting machine learning algorithm that adds trees sequentially with each one learning from the preceding tree. As well, with each tree that's being built it fits the residuals of the model's loss which corrects any errors that previous ones may have made which leads to a more accurate model.

F. Libraries

Python, Scikit-learn, XGBoost, Pandas, NumPy, and Matplotlib are several libraries we will be using. This list is not exclusive to the libraries mentioned and will consider the use of other libraries if necessary.

- Python will serve as the primary programming language for the entire project due to its simplicity, readability, and its plethora of data science libraries to help with the machine learning process.
- NumPy (Numerical Python) serves to allow our program to perform high-performance mathematical operations. Due to the nature of the project, it is almost required for the intense number of calculations and more importantly Scikit-learn and XGBoost internally rely on NumPy arrays for data handling.
- Pandas will be used for data manipulation and analysis. In this particular project, our dataset (PhiUSIIL) is tabular in which Pandas offers a data structure called DataFrames which will efficiently handle the data from PhiUSIIL ranging from loading the dataset to handling any inconsistent or missing data.
- Scikit-learn is a machine learning library that Python offers. In this project, Scikit-learn will allow our group to implement Random Forest and compute evaluation metrics. Scikit-learning is important since this will allow a fair comparison against XGBoost.
- Random Forest library from Sklearn allows for the classification of Random Forest algorithms, granting access to estimators to build trees, random state, and n-jobs.
- XGBoost library from Sklearn provides a highly optimized version of the gradient boosting algorithm. Some important features which will be heavily used throughout the project will be feature ranking and model training and tuning.
- Matplotlib is a visualization library that we will use to generate graphics to show our results.
- Time is a library that allows a function to measure the computational efficiency of the Random Forest and XGBoost's inference and training time.
- TfidfVectorizer is a library that allows the use of text features by transforming raw text into numerical TF-IDF feature vectors.

A. Environments

To develop such a program, we will primarily use Google Colab

- Google Colab offers a cloud-based alternative to Jupyter Notebook. Google Colab also handles computationally intensive tasks which will help in our model training of XGBoost. Lastly, Google Colab will allow our group to share execute code with little hassle.

III. Methodology

A. Research Question

The primary research question of this project is which algorithm, XGBoost or Random Forest, performs better in URL phishing detection. This question will be addressed through three evaluation criteria which include detection of accuracy, practicality, and overall model robustness.

B. Datasets

The primary collection of data for this project comes from the UCI PhiUSIIL Phishing URL dataset. [2] This dataset is composed of 235,795 total instances (URLs and their corresponding webpages), with 134,850 being the benign or "legitimate" URLs while the rest of the 100,945 instances are phishing URLs. The associated tasks are classification, which is perfect for the machine learning algorithms being used since both XGBoost and Random Forest are considered tree-based classifiers. The dataset utilizes labeling to differentiate the URLs with label 1 being legitimate and label 0 being phishing. This mapping that the dataset uses is instrumental in interpreting the data and ensures that precision and recall are evaluated correctly.

The second collection of data comes from Kaggle, which houses a library of publicly available datasets made by contributors in the community. A second dataset was used to scope just how well the models trained and how well it would generalize new data when introduced to a new dataset. [3] The dataset comprises a total of 11,430 URL instances with half of them being labeled as legitimate and the other half being labeled as phishing. It has 87 different extracted features with all but one being numeric features. The only text feature in the dataset was "url". This dataset also uses labeling to differentiate the URLs with string labels such as "legitimate" or "phishing". To keep testing consistent with the first dataset, label mapping was used to match the string labels with numeric labels mapped to 0 for legitimate and 1 for phishing.

These two datasets will serve as our standard of measurement for both XGBoost and Random Forest. They will train using both the benign and phishing URLs gathered from both the datasets, and their performances will be analyzed to determine which one is the more optimal model.

C. Preprocessing & Feature Selection

Before the dataset could be used for model training, it needs to go through some vigorous cleaning methods to ensure that the data is consistent and reliable to test and train on.

This was done by first removing whitespace from the column labels to avoid any data that could be misinterpreted. The next step in the preprocessing method was to map all infinite values to NaN, and subsequently drop all the NaN values to avoid any disparities in the data with infinite values.

For the feature selection, a filter-based feature selection was used for training and testing. This was done because the dataset has 54 features so simplifying the dataset, at least for now, is optimal for current training and testing. This was done by utilizing the SelectKBest method with the ANOVA F-statistic to rank the top 20 features of the dataset and utilize the features to train and test the data accordingly. The F-score assesses the disparity between both benign (label 1) and phishing (label 0) classes within each feature.

For consistency, the text features were dropped and only the numeric features were kept. The 20 features with the highest-ranking features were retained for model development.

D. Model Training

For model training, an XGBoost classifier utilized 100 trees on the baseline model and 200 trees on the combined features model to give the model more learning capacity for the higher-dimensional input space. Each version was trained the same on each dataset with hyperparameters such as depth of 8, learning rate of 0.1, and subsample of 0.8. Performance was also evaluated using the same standard metrics, including accuracy, precision, recall, and f1-score.

For model training, a Random Forest classifier with 100 trees was trained on each dataset. Two versions were evaluated, the first being numerical features only which served as our baseline, and the second version being the combination of numerical and the TF-IDF text representations. Performance was evaluated using standard metrics which include accuracy, precision, recall, and f1-score.

E. 5-Fold Cross Validation

Cross-Validation was utilized as a new method of evaluation of the model's performance. Most notably, to test the robustness of the model. To obtain a more reliable and robust evaluation, the k-fold (5-fold was employed) cross validation procedure was implemented. The two models were trained on four folds and evaluated on the remaining fold. The accuracy scores were then given from the 5-folds, and the mean and standard deviation were taken to quantify the overall robustness of the model.

IV. Evaluation Framework

The evaluation of the results involves taking the results from each test and using the values to calculate accuracy, precision, recall, and F1-scores of the LLMs. The overall results will be used to determine the robustness of the models and their inference/training times.

A. Accuracy

The accuracy of the learning language models, XGBoost and Random Forest, are determined using the accuracy_score function. The overall mean accuracy takes the accuracies from each 5-Fold Cross-Validation of the LLMs with each of the datasets.

B. Precision

The precision of each trial will be measured to determine their level of performance in identifying phishing and benign data. The more each model accurately predicts a phishing attempt over the total number of URLs checked (True Positives/True Positives + False Positives), the higher their precision increases. The two measured precisions will then be used to determine the macro and weighted average of the LLMs after each run with the datasets.

C. Recall

The recall of the LLMs is an important measurement of the project, as it will be one of the primary indicators of the models and their ability to accurately determine phishing attacks through the URLs. Measured by dividing the number of each model's true positives over the sum of true positives and false negatives, the recall value will give insight into the models' ability to accurately identify true positive URLs.

D. F1-Score

The F1 score takes the models' precision and recall values in order to score how efficiently each model is at identifying positive scenarios; the harmonic mean.

E. Robustness

To test robustness, Gaussian noise was added by the features standard deviation and scaled in levels of 0.01, 0.05, and 0.10. Random Forest remained stable as noise increased while XGBoost struggled heavily in comparison. XGBoost overfit the data in dataset 1, which revealed its weaknesses in generalization and sensitivity to perturbations.

F. Inference

The inference of the models is their training to examine and determine the difference between analyzed pieces of data. In the case of the Random Forest and XGBoost models, their inference training involves the time it takes to look over and assign values to data.

V. Results

The following results examine how Random Forest and XGboost both performed in detecting phishing URL's given the baseline model and combined features model for both algorithms. The 5-fold cross-validation reports, robustness results, as well as the computational efficiency results are also highlighted given each of the model's performances.

A. Random Forest

Random Forest's performance on the first dataset yielded near perfect results with extremely high accuracy, precision, recall, and f1-score on both the baseline model (numeric features only) and the combined features model (numeric and text features). The combined features model performed marginally better with only a slight increase in performance in recall.

The 5-fold cross-validation was applied to both models to see the estimation of the model's performance and reduce overfitting. This also yielded a near perfect mean accuracy and had zero variance across all folds.

```
=== Random Forest (Top-20 features) ===
Accuracy: 0.9999
              precision    recall  f1-score   support

 Phishing(0)     1.0000    0.9997    0.9999     30284
   Benign(1)     0.9998    1.0000    0.9999     40455

    accuracy                         0.9999     70739
   macro avg     0.9999    0.9999    0.9999     70739
weighted avg     0.9999    0.9999    0.9999     70739


 ------------------------------------------------------


=== Random Forest (Combined Features) ===
Accuracy: 0.9999
              precision    recall  f1-score   support

 Phishing(0)     1.0000    0.9998    0.9999     30284
   Benign(1)     0.9999    1.0000    0.9999     40455

    accuracy                         0.9999     70739
   macro avg     0.9999    0.9999    0.9999     70739
weighted avg     0.9999    0.9999    0.9999     70739
```

```
5-Fold Cross-Validation — Random Forest (Baseline)

Accuracy scores: [0.9999 0.9999 0.9999 0.9999 0.9999]
Mean Accuracy: 0.9999
Standard Deviation: 0.0000

------------------------------------------------------------

5-Fold Cross-Validation — Random Forest (Combined Features)

Accuracy scores: [0.9999 0.9999 1.     0.9999 0.9999]
Mean Accuracy: 0.9999
Standard Deviation: 0.0000
```

Figures 2 – 3. Random Forest's Results and Cross Validation for Dataset 1

While not inherently bad in itself, having a model that performs this well indicates that the data may be too clean and even a little unrealistic in comparison to real-world data. This was a main factor in introducing a new dataset to the models to see how well it would generalize new data.

Using the same configurations utilized in dataset 1, dataset 2 was also split into two models, the baseline and combined features as well as implemented the same 5-fold cross-validation. This was done to get a stronger test of the model's generalization as well as robustness.

The results show that there is more variation between results, with both Benign and Phishing classes sitting around the 96 percentile and a more drastic increase in the addition of the combined features in comparison to the first dataset which only saw an increase of 0.0001 in recall. This dataset across the board performed better with the addition of the text features to the numeric data. Though not extreme, it is consistent which suggests that the text patterns in this dataset's URLs provided more distinction in detecting malicious URLs

in comparison to the first dataset where the text features did not add much to the model.

```
=== Random Forest Dataset 2 (Top-20 features) ===
Accuracy: 0.9609
              precision    recall  f1-score   support

 Phishing(0)     0.9593    0.9627    0.9610      1715
   Benign(1)     0.9625    0.9592    0.9608      1714

    accuracy                         0.9609      3429
   macro avg     0.9609    0.9609    0.9609      3429
weighted avg     0.9609    0.9609    0.9609      3429


 ------------------------------------------------------

=== Random Forest Dataset 2 (Combined Features) ===
Accuracy: 0.9665
              precision    recall  f1-score   support

           0     0.9635    0.9697    0.9666      1715
           1     0.9695    0.9632    0.9663      1714

    accuracy                         0.9665      3429
   macro avg     0.9665    0.9665    0.9665      3429
weighted avg     0.9665    0.9665    0.9665      3429
5-Fold Cross-Validation — Random Forest Dataset 2 (Baseline)

Accuracy scores: [0.9593 0.9637 0.9584 0.9619 0.9654]
Mean Accuracy: 0.9618
Standard Deviation: 0.0026

 ------------------------------------------------------------

5-Fold Cross-Validation — Random Forest Dataset 2 (Combined Features)

Accuracy scores: [0.9619 0.9646 0.9663 0.9637 0.9711]
Mean Accuracy: 0.9655
Standard Deviation: 0.0031
```

Figures 4 – 5. Random Forest's Results and Cross Validation for Dataset 2

The mean accuracy was 0.9618 with a variance of 0.0026 in the baseline model while the combined features model had a mean accuracy of 0.9655 and a variance of 0.0031. Though the models didn't perform as well as the first dataset, it is more reflective of real-world complexity and still has exceptional stability based on the low variance.

B. XGBoost

The XGBoost models were configured in the exact same way as the Random Forest models to generate a fair comparison between the two models. This can be seen in the results of XGBoost because it had the exact same results as random forest in accuracy, precision, recall, and f1-score. The only difference is the mean accuracy in the 5-fold cross-validation test where the mean accuracy was a perfect score. However, this is a miniscule difference since the results for Random Forest were already near perfect.

```
=== XGBoost (Baseline) ===
Accuracy: 0.9999
              precision    recall  f1-score   support

 Phishing(0)     1.0000    0.9997    0.9999     30284
   Benign(1)     0.9998    1.0000    0.9999     40455

    accuracy                         0.9999     70739
   macro avg     0.9999    0.9999    0.9999     70739
weighted avg     0.9999    0.9999    0.9999     70739


------------------------------------------------------------

=== XGBoost (Combined Features) ===
Accuracy: 0.9999
              precision    recall  f1-score   support

 Phishing(0)     1.0000    0.9998    0.9999     30284
   Benign(1)     0.9999    1.0000    0.9999     40455

    accuracy                         0.9999     70739
   macro avg     0.9999    0.9999    0.9999     70739
weighted avg     0.9999    0.9999    0.9999     70739
```

```
5-Fold Cross-Validation — XGBoost (Baseline)

Accuracy scores: [1.     0.9999 0.9999 0.9999 0.9999]
Mean Accuracy: 0.9999
Standard Deviation: 0.0000

------------------------------------------------------------

5-Fold Cross-Validation — XGBoost (Combined Features)

Accuracy scores: [1.     0.9999 1.     1.     0.9999]
Mean Accuracy: 1.0000
Standard Deviation: 0.0000
```

Figures 6 – 7. XGBoost's Results and Cross Validation for Dataset 1

Based on the findings in the first dataset, these results are to be expected since the dataset is very clean with very little noise in it. The results in the second dataset were a higher indicator of how well the models performed.

XGBoost had lower performance in the second dataset, however it is a lot more realistic given the results. Comparing the results of both Random Forest and XGBoost, this dataset paints a more accurate picture of how well they were able to reliably train and evaluate the data. XGBoost had better accuracy, precision, recall, and f1-score, though not by an extreme amount, however the difference is there. In the 5-fold cross-validation test the mean accuracy was also greater than that of random forest, with a lower variance which means that XGBoost was more stable than Random Forest. However, given these results they are not a huge factor since the results are so close in scope. The following metrics will tell a bigger picture of which model handles data better.

```
=== XGBoost Datset 2 (Baseline) ===
Accuracy: 0.965
              precision    recall  f1-score   support

 Phishing(0)     0.9650    0.9650    0.9650      1715
   Benign(1)     0.9650    0.9650    0.9650      1714

    accuracy                         0.9650      3429
   macro avg     0.9650    0.9650    0.9650      3429
weighted avg     0.9650    0.9650    0.9650      3429


------------------------------------------------------------

=== XGBoost Dataset 2 (Combined Features) ===
Accuracy: 0.967
              precision    recall  f1-score   support

 Phishing(0)     0.9679    0.9662    0.9670      1715
   Benign(1)     0.9662    0.9679    0.9671      1714

    accuracy                         0.9670      3429
   macro avg     0.9670    0.9670    0.9670      3429
weighted avg     0.9670    0.9670    0.9670      3429

5-Fold Cross-Validation — XGBoost Dataset 2 (Baseline)

Accuracy scores: [0.9624 0.9668 0.9663 0.9628 0.9703]
Mean Accuracy: 0.9657
Standard Deviation: 0.0029

------------------------------------------------------------

5-Fold Cross-Validation — XGBoost Dataset 2 (Combined Features)

Accuracy scores: [0.9633 0.9668 0.9637 0.9659 0.9703]
Mean Accuracy: 0.9660
Standard Deviation: 0.0025
```

Figures 8 – 9. XGBoost's Results and Cross Validation for Dataset 2

C. Robustness Results

Robustness is an important metric to test for because it shows how each model handles data when it's not perfect or when It's been perturbed. To assess the robustness, Gaussian noise was added at three levels: 0.01, 0.05, and 0.1) for both of the datasets. The changes in accuracy are an indicator for just how well the models can manage adversarial variations.

In the first dataset, Random Forest was shown to handle noise extremely well with an accuracy of 0.9999 and 0.9970 at 0.01 and 0.05 respectively. However, it contained a dip in the latter half with an accuracy of 0.9495.

The overall results of Random Forest in this first dataset were still adequate and at times extremely good. When it comes to XGBoost though, the model handled the noise very poorly. At noise levels 0.01, 0.05, and 0.1 it held an accuracy of about 0.57. This is not optimal at all considering that even with the smallest adjustment of noise, it collapses immediately. This shows that XGBoost cannot handle fluctuations in data and the model is extremely sensitive to noise. A factor of this could be that XGBoost was memorizing patterns rather than generalizing them, which would explain the incredibly low accuracy score when noise is introduced.

```
=== Random Forest Robustness Evaluation ===
Noise Level: 0.01 | Accuracy = 0.9999
Noise Level: 0.05 | Accuracy = 0.9970
Noise Level: 0.10 | Accuracy = 0.9495

=== XGBoost Robustness Evaluation ===
Noise Level: 0.01 | Accuracy = 0.5716
Noise Level: 0.05 | Accuracy = 0.5711
Noise Level: 0.10 | Accuracy = 0.5702
```

Figure 10. Random Forest and XGBoost Robustness Dataset 1

For dataset 2, Random Forest didn't perform as well as in dataset 1, however it still had very high accuracy across all noise levels with accuracies of 0.9612, 0.9621, and 0.9565 at noise levels 0.01, 0.05, and 0.1 respectively.

In contrast, XGBoost ended up performing vastly better in the second dataset. It had decent accuracy scores with 0.9119, 0.9134, and 0.9029 at noise levels 0.01, 0.05, and 0.1 respectively. Though not as good at Random Forest at resisting noise, it still is an immense improvement over the former results in the first dataset.

Overall, Random Forest is the more robust model than XGBoost given the resistance to noise and more generalizable, stable decision boundaries that are able to withstand the perturbations added to the original datasets.

```
=== Random Forest Robustness Evaluation ===
Noise Level: 0.01 | Accuracy = 0.9612
Noise Level: 0.05 | Accuracy = 0.9621
Noise Level: 0.10 | Accuracy = 0.9565

=== XGBoost Robustness Evaluation ===
Noise Level: 0.01 | Accuracy = 0.9119
Noise Level: 0.05 | Accuracy = 0.9134
Noise Level: 0.10 | Accuracy = 0.9029
```

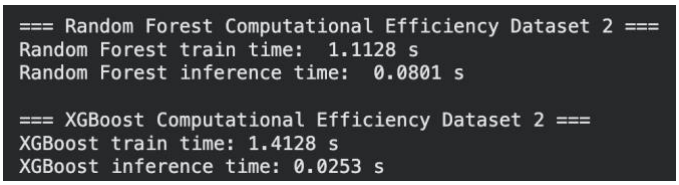
Figure 11. Random Forest and XGBoost Robustness Dataset 2

D. Computational Efficiency Results

The computational efficiency of the models measures how effective they are at computing the datasets and making determinations with each piece of data scanned.

```
=== Random Forest Computational Efficiency ===
Random Forest train time:  14.8320 s
Random Forest inference time:  0.6121 s

=== XGBoost Computational Efficiency ===
XGBoost train time: 5.9798 s
XGBoost inference time: 0.2860 s
```



Figures 12 – 13. Random Forest and XGBoost Computational Efficiency for Dataset 1 and 2

Overall, the XGBoost LLM was more efficient at evaluating datasets than the Random Forest with a shorter inference time of 0.0253s when compared to that of the Random Forest's 0.0.801s, both for the second dataset. For the first set, the pattern repeated with XGBoost having a time of 0.2860s and Random Forest having a time of 0.6121s.

Additionally, the training time needed for the two models varied, as the Random Forest had a slightly shorter time of 1.1128s for dataset 2 when compared to the XGBoost's time of 1.4128s. However, the former also took much longer to train for a first dataset at 14.832s compared to the latter's 5.9798.

VI. Conclusions

In the first dataset, both Random Forest and XGBoost performed incredibly similarly. The second dataset saw more variation since both had a performance drop. However, XGBoost had better accuracy than Random Forest.

In terms of robustness, Random Forest was the superior model since it was able to handle noise with very little fluctuations while XGBoost performance dropped, especially given the first dataset. For computational efficiency, XGBoost was superior in how much more efficient it was at training and inference time.

Though XGBoost was more computationally effective and performed equally with Random Forest, even at times outperforming it such as in the second dataset. Overall, Random Forest is the superior model based on how well it handled noise. Attackers will always find a way to compromise systems, and data will not always be clean enough to yield the best accuracy in detecting attacks.

Having a robust model that can still perform well and mitigate some of the dangers of having imperfections in data is a very strong trait to have. XGBoost was able to handle it fairly well in the second dataset, but the drastic drop in performance in the first dataset causes major problems.

Given that Random Forest was able to handle noise extremely well in both datasets and XGBoost's accuracy was miniscually better than that of Random Forest, it is not a strong enough jump in accuracy performance to warrant being considered better than Random Forest. Therefore, based on the findings in this paper, Random Forest proved to be the superior model in phishing URL detection.

References

[1] [6] U.S. Department of State, https://www.state.gov/understanding-and-preventing-phishing-attacks/ (accessed Sep. 23, 2025).

[2] A. Prasad and S. Chandra. "PhiUSIIL Phishing URL (Website)," UCI Machine Learning Repository, 2024. [Online]. Available: https://doi.org/10.1016/j.cose.2023.103545.

[3] Hannousse, Abdelhakim; Yahiouche, Salima (2021), "Web page phishing detection", Mendeley Data, V3, doi: 10.17632/c2gw7fy2j4.3 Available: https://www.kaggle.com/datasets/shashwatwork/web-page-phishing-detection-dataset?resource=download

[4] D. N, E. P. Sudarsan, B.Praveen, T. D, and S. K, "Enhanced phishing detection: Integrating random forest classifier and domain analysis for proactive cybersecurity," *2024 8th International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC)*, pp. 633–643, Oct. 2024. doi:10.1109/i-smac61858.2024.10714859

[5] L. Jovanovic *et al.*, "Improving phishing website detection using a hybrid two-level framework for feature selection and XGBoost tuning," *Journal of Web Engineering*, Jul. 2023. doi:10.13052/jwe1540-9589.2237